Alberto Clocks

Developer's Guide

Time is Luxury

Instructor.	Sir M. Arqum Tariq	
Batch.	2606G2	
Group.	D	
Serial No.	**Enrollment Number**	**Student Name**
1.	1732077	Fatima Raza
2.	1732701	Alishah Sabri
3.	1733431	Areeba Sabri

Aptech eProject

Contents

1 Introduction **5**

1.1 Main Features 5

1.2 Project Limitations 5

2 Project Requirements **6**

2.1 Software Requirements 6

2.2 Hardware Requirements 6

2.3 Functional Requirements 7

2.4 Non-functional Requirements 7

3 How to Run the Project **8**

3.1 Opening the Product Details Page 8

4 Project Folder Structure **8**

4.1 HTML Files 10

4.2 CSS Files and Their Uses 10

4.3 JavaScript Files and Their Uses 11

4.4 Product Data, Images and Fonts 11

5 How the Website Works **12**

5.1 HTML Documents 12

5.2 CSS Layer 12

5.3 JavaScript Modules 12

5.4 External Resources 12

5.5 Browser Storage 13

6 Script Load Order **13**

7 Shared Functions **14**

8 Product Data **14**

8.1 Adding a New Product 15

8.2 Loading the Catalogue 15

8.3 Search and Filters 15

8.4 Image Fallback 16

9 Product Details Page **17**

10 Wishlist and Reviews **18**

10.1 Wishlist Module 18

10.2 Reviews Module 19

11 Login and Sign-Up **20**

11.1 Registration Checks 20

11.2 Password Rules 20

11.3 Successful Registration 22

11.4 Incorrect Login Details 22

11.5 Password Error 22

11.6 Logging Out 23

11.7 Account Limitation 23

12 Profile and Order History **23**

12.1 My Profile 23

12.2 Order History 24

13 Shopping Cart and Coupon **24**

13.1 Adding a Product 24

13.2 Changing Quantity 24

13.3 Coupon 25

14 Checkout Implementation **26**

14.1 Checkout Fields 26

14.2 Phone Validation 26

14.3 Saving an Order 27

14.4 Checkout Flow 28

14.5 Checkout Limitation 29

15 Clock, Ticker and Themes **29**

15.1 World Time 29

15.2 Store Status 29

15.3 Location 29

16 Support and Newsletter Forms **30**

17 Browser Storage **31**

17.1 Resetting Demo Data 31

18 Risks and Limitations **32**

19 Troubleshooting **33**

20 Testing Checklist **34**

21 Maintenance and Future Improvements **34**

21.1 Changing Product Information 34

21.2 Changing the Design 35

21.3 Changing a Feature 35

21.4 Possible Future Improvements 35

22 Conclusion **35**

List of Figures

1 Home page of Alberto Clocks 6

2 How the parts of Alberto Clocks work together 12

3 Product cards and filter controls 16

4 Product cards and filter controls 16

5 Full product details page 17

6 Wishlist panel 18

7 Review list and rating form 19

8 Login window 20

9 Sign-up form 21

10 Password checklist 21

11 Incorrect login message 22

12 Password validation message 22

13 User profile and account details 23

14 Order history and order status timeline 24
15 Shopping cart with items and quantity controls 25
16 Coupon field and updated total 25
17 Checkout form and order summary 27
18 Checkout steps 28
19 Clock, ticker, themes and store status 29
20 Support section and contact form 30
21 Store Locator and map 30

1. Introduction

Alberto Clocks is a front-end website for displaying wall, luxury and vintage clocks. Visitors can explore products, read product details, use a shopping cart, create a demonstration account and view store information.

The project uses HTML for page structure, CSS for appearance, JavaScript for interactive features and a JSON file for product information.

This guide explains how the project is arranged, how the main features work, how to run it, how to test it and what to check when changing the code.

1.1 Main Features

- Home section with a digital clock and date.
- Store opening status and ticker.
- World-time cards for selected cities.
- Dark/light theme and LED colour controls.
- Product catalogue with 24 clock records.
- Search, category filters, sorting and price slider.
- Product cards and a full product details page.
- Shopping cart, quantities and coupon.
- Wishlist and product reviews.
- Registration, login, profile and log out.
- Password rules and form validation.
- Checkout demonstration and order history.
- Store Locator, FAQs and contact form.
- Responsive layout for smaller screens.

1.2 Project Limitations

This project is made for learning and demonstration. Accounts, cart items, wishlist entries, reviews, contact messages and orders are saved in the current browser.

The website does not take real payments, send orders to a store, send real contact emails or show live courier updates. The order status is a demonstration. Saved information is not shared between devices.

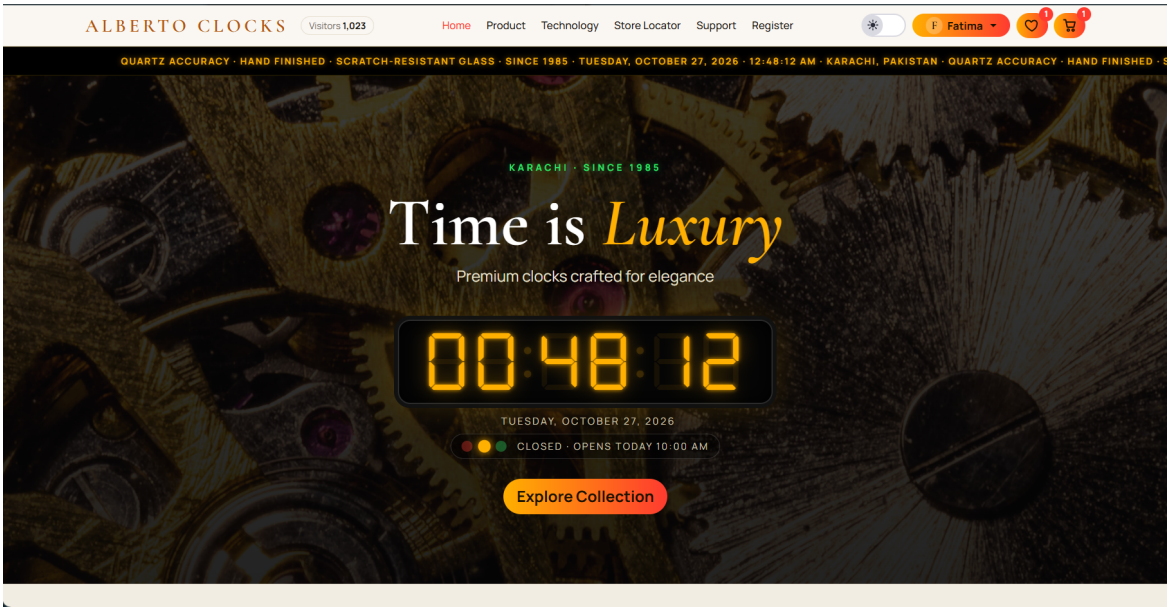


Figure 1: *Home page of Alberto Clocks*

2. Project Requirements

2.1 Software Requirements

- Operating system: Windows, macOS or Linux.
- Code editor: Visual Studio Code or another text editor.
- Browser: current Chrome, Edge, Firefox or Safari.
- Local server: VS Code Live Server or another HTTP server.
- Technologies: HTML5, CSS3, JavaScript, Bootstrap and JSON.
- Internet connection for remote product photographs, the map and the configured checkout connection check.

2.2 Hardware Requirements

A standard computer or laptop that can run a modern browser and code editor is sufficient. A phone or tablet can be used to test the mobile layout.

2.3 Functional Requirements

ID	Requirement	Main Area
FR-01	Display the Home, Product, Technology, Store Locator, Support and Register sections.	`index.html`
FR-02	Load product information from JSON.	Product module
FR-03	Search, sort, filter and move through product pages.	Product module
FR-04	Display product pictures, descriptions, specifications and stock.	Detail page
FR-05	Add products to the cart and change quantities.	Cart module
FR-06	Add or remove wishlist items.	Wishlist module
FR-07	Register, log in, log out and view profile details.	Account module
FR-08	Check password rules and required form fields.	Forms module
FR-09	Demonstrate checkout and save an order in the browser.	Checkout
FR-10	Show order history and demonstration status.	Profile
FR-11	Display product ratings and reviews.	Review module
FR-12	Display the clock, ticker, themes and store status.	UI module
FR-13	Display store information, map, FAQs and contact form.	Store / Support

2.4 Non-functional Requirements

Quality	Requirement
Ease of Use	Navigation, buttons, messages and forms should be easy to understand.
Responsive Design	Pages should fit desktop, tablet and mobile screens.
Browser Support	The website should work in current mainstream browsers.
Maintainability	Product information, CSS and JavaScript should stay in their own files.
Reliability	Missing data or images should show a message or fallback.
Accessibility	Forms should have labels and keyboard focus should be visible.
Privacy	Location is permission-based and saved data remains in the current browser.

3. How to Run the Project

The website loads `data/products.json` using JavaScript. Opening `index.html` directly from the file system can stop the product data from loading. Use a local server when testing.

1. Open the E-project folder in Visual Studio Code.
2. Install Live Server if needed.
3. Right-click `index.html`.
4. Select **Open with Live Server**.
5. Check that the Home page and product collection appear.

If the product list is empty, check the JSON file path, file format and browser console.

3.1 Opening the Product Details Page

The product details page is `pages/product.html`. It uses a product ID in the page address. For example:

`pages/product.html?id=1`

The ID tells the page which clock to display.

4. Project Folder Structure

The following is the project folder structure.

```
E-project/
|-- index.html
|-- README.md
|
|-- css/
|   |-- about.css
|   |-- accessibility.css
|   |-- auth.css
|   |-- base.css
|   |-- bootstrap.min.css
|   |-- cart-footer.css
|   |-- features.css
|   |-- forms-faq.css
|   |-- header.css
|   |-- hero-bg.css
```

```
|   |-- hero.css
|   |-- led.css
|   |-- pages.css
|   |-- product-photos.css
|   |-- products.css
|   |-- responsive.css
|   |-- sections.css
|   |-- store-status.css
|   |-- style.css
|   |-- theme-toggle.css
|
|-- data/
|   |-- products.json
|
|-- fonts/
|   |-- DSEG7Classic-Regular.woff2
|   |-- README.txt
|
|-- image/
|   |-- about-mechanism.svg
|   |-- about-wall-clock.svg
|   |-- favicon.svg
|   |-- hero-bg.svg
|   |-- register-dial.svg
|   |-- products/
|       |-- clock-1.svg ... clock-24.svg
|
|-- js/
|   |-- auth.js
|   |-- bootstrap.bundle.min.js
|   |-- cart.js
|   |-- core.js
|   |-- forms.js
|   |-- main.js
|   |-- orders.js
|   |-- products.js
|   |-- reviews.js
|   |-- security.js
|   |-- ui.js
|   |-- wishlist.js
|
|-- pages/
    |-- product.html
```

4.1 HTML Files

- `index.html`: main page and website sections. It also contains the interface elements for login, cart, wishlist, checkout and profile.
- `pages/product.html`: full details page for a selected clock.
- `README.md`: project notes, folder overview and settings.

4.2 CSS Files and Their Uses

The file `css/style.css` imports the other CSS files. Keep the import order because a later file can change a rule from an earlier file.

CSS File	**Purpose**
`about.css`	Styles the About and Technology content.
`accessibility.css`	Skip link, keyboard focus and reduced-motion styles.
`auth.css`	Login and account forms.
`base.css`	Basic layout, fonts and shared rules.
`bootstrap.min.css`	Bootstrap layout and component styles.
`cart-footer.css`	Cart panel and footer.
`features.css`	Extra features such as wishlist, stock labels, ratings, price slider, world time, coupon and checkout.
`forms-faq.css`	Forms and FAQ styles.
`header.css`	Header and navigation.
`hero-bg.css`	Home background image rules.
`hero.css`	Home hero section.
`led.css`	LED display and colour controls.
`pages.css`	Product details and other page styles.
`product-photos.css`	Product photo presentation.
`products.css`	Product cards and catalogue.
`responsive.css`	Mobile and smaller-screen layout.
`sections.css`	Main website sections.
`store-status.css`	Store opening-status display.
`style.css`	Imports the other CSS files.
`theme-toggle.css`	Dark/light theme switch.

4.3 JavaScript Files and Their Uses

JavaScript File	Purpose
`auth.js`	Registration, login, profile and log out.
`bootstrap.bundle.min.js`	Bootstrap menus, tabs, modals and components.
`cart.js`	Cart items, quantities, coupon and checkout demonstration.
`core.js`	Shared functions, state and browser storage helpers.
`forms.js`	Password rules, contact form and newsletter form.
`main.js`	Starts the main website features.
`orders.js`	Demo order history and status timeline.
`products.js`	Loads JSON, builds product cards and opens details.
`reviews.js`	Product reviews and ratings.
`security.js`	Password-related helper functions.
`ui.js`	Clock, ticker, themes, navigation and store status.
`wishlist.js`	Adds and removes wishlist items.

4.4 Product Data, Images and Fonts

The file `data/products.json` contains product IDs, names, categories, prices, descriptions, specifications, image paths and stock values.

The folder `image/products/` contains the clock illustrations `clock-1.svg` to `clock-24.svg`. Other SVG files are used for the Home background, About section, registration design and favicon.

The file `fonts/DSEG7Classic-Regular.woff2` is used for the digital clock. The file `fonts/README.txt` explains how optional fonts can be added for offline use.

5. How the Website Works

HTML provides the content and structure. CSS controls appearance. JavaScript handles the actions such as searching, cart updates and login. The product list is read from JSON. Browser storage keeps demonstration information between page reloads.

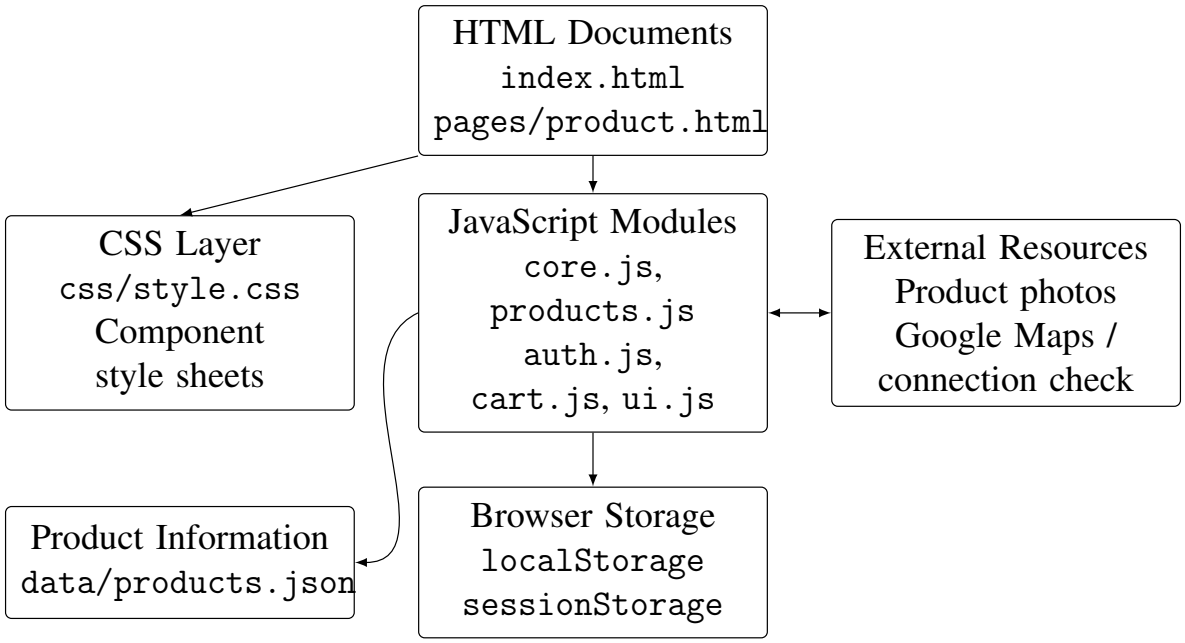


Figure 2: *How the parts of Alberto Clocks work together*

5.1 HTML Documents

The two HTML documents provide the main website and the full product details page. They contain the layout, buttons, forms and panels used by the JavaScript modules.

5.2 CSS Layer

CSS controls fonts, spacing, product cards, forms, navigation, themes and responsive layout. The main CSS file imports the other stylesheets.

5.3 JavaScript Modules

JavaScript reads product data and responds to user actions. It updates the product list, cart, wishlist, account forms and other parts of the page.

5.4 External Resources

Some product pictures and map content use remote resources. They may not appear without an Internet connection. Checkout also checks the connection before opening the form.

5.5 Browser Storage

The cart, wishlist, demo accounts, reviews, messages and orders are stored in the current browser. These records are not shared between devices.

6. Script Load Order

Script load order means the order in which JavaScript files are loaded. Some files provide functions used by later files, so their order should not be changed without checking their dependencies.

Given below is the reference table.

Order	Script and Purpose
1	`bootstrap.bundle.min.js` – Bootstrap menus, tabs and dialogs.
2	`core.js` – shared functions, state and storage helpers.
3	`security.js` – password-related helper functions.
4	`wishlist.js` – wishlist actions.
5	`reviews.js` – review display and ratings.
6	`products.js` – product data, cards and product details.
7	`orders.js` – demonstration order status and history.
8	`cart.js` – cart, coupons and checkout.
9	`auth.js` – registration, login and profile.
10	`forms.js` – password rules and contact/newsletter forms.
11	`ui.js` – clock, ticker, themes, navigation and store status.
12	`main.js` – starts the website after the other modules load.

7. Shared Functions

The file `js/core.js` contains functions used by different parts of the website.

- `\$()` finds one element on the current page.
- `\$\$()` finds a list of elements.
- `store.get()` reads saved JSON data from browser storage.
- `store.set()` saves a value in browser storage.
- `esc()` escapes text before it is added to HTML.
- `money()` formats a price in Pakistani rupees.
- `toast()` displays a short message after an action.
- Bootstrap helper functions open dialogs and side panels.

The shared state includes the product list, cart and active user. The feature modules use these values to update the screen.

8. Product Data

The catalogue is stored in `data/products.json`. It contains 24 clocks in the Wall, Luxury and Vintage categories.

Field	**Purpose**
`id`	Unique number used to identify a clock.
`name`	Product name shown on the card and detail page.
`cat`	Category key: wall, luxury or vintage.
`price`	Numeric price in rupees.
`desc`	Product description.
`spec`	Specifications shown on the detail page.
`img`	Main product image address.
`fallback`	Local SVG image used if the main image fails.
`stock`	Starting stock value for the demonstration.

8.1 Adding a New Product

1. Give the clock a unique ID.
2. Add the name and category.
3. Set the price as a number.
4. Add a description and specifications.
5. Add the main image address.
6. Add a valid fallback image path.
7. Add a stock value.
8. Check that the JSON format is valid.

After editing, open the website through Live Server and test the product card, details page, cart and wishlist.

8.2 Loading the Catalogue

The product module requests the JSON file. Temporary skeleton cards may appear while it loads. If loading fails, the website displays an error message instead of leaving an unexplained empty collection.

8.3 Search and Filters

The product collection shows six clocks per page. Search, category, sorting and maximum price are applied before the visible cards and page controls are updated.

8.4 Image Fallback

The website tries the main image first, then the local fallback and finally a drawn clock face. This helps prevent a missing remote image from leaving a blank card.

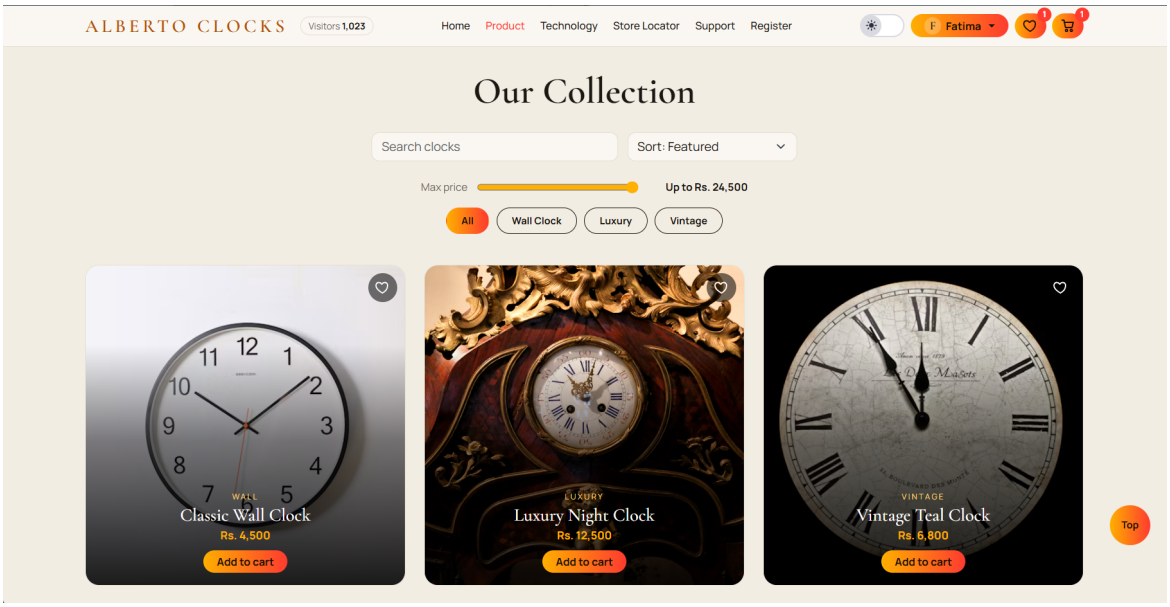


Figure 3: *Product cards and filter controls*

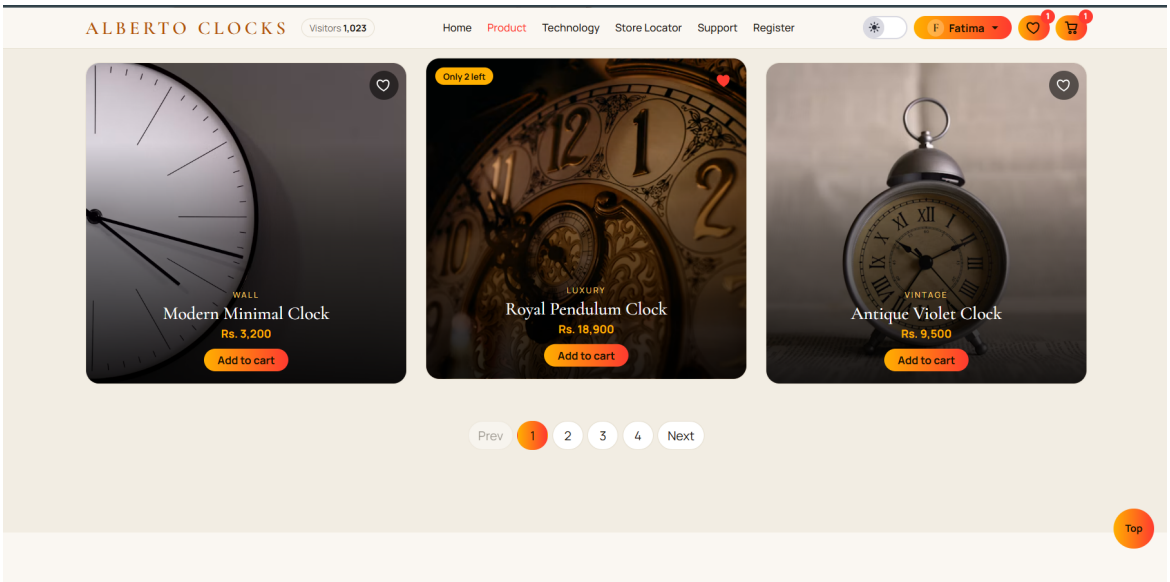


Figure 4: *Product cards and filter controls*

9. Product Details Page

The detail page reads a product ID from the page address. For example, `pages/product.html?id=5` opens the clock with ID 5.

The page displays the product name, image, description, price, stock, specifications, related clocks and reviews. It also contains quantity controls and a wishlist button.

If the ID does not match a product, the page shows a not-found message and a way to return to the product collection.

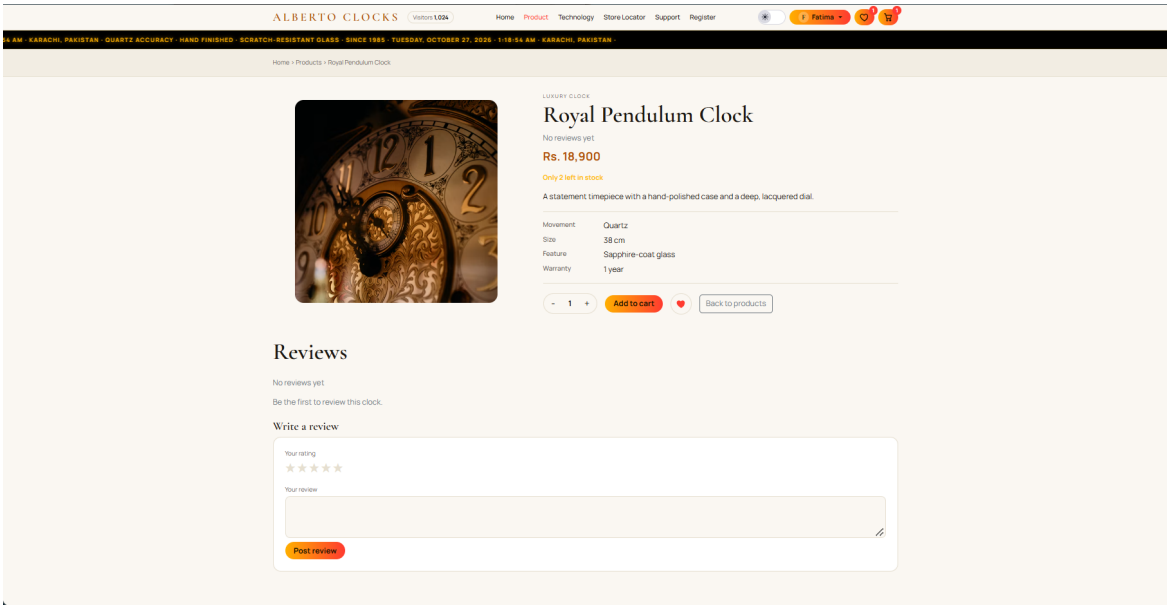


Figure 5: *Full product details page*

10. Wishlist and Reviews

10.1 Wishlist Module

The file `js/wishlist.js` saves product IDs in browser storage. When an item is added or removed, the heart buttons, wishlist count and saved-item panel are updated.

Wishlist items do not reserve stock.

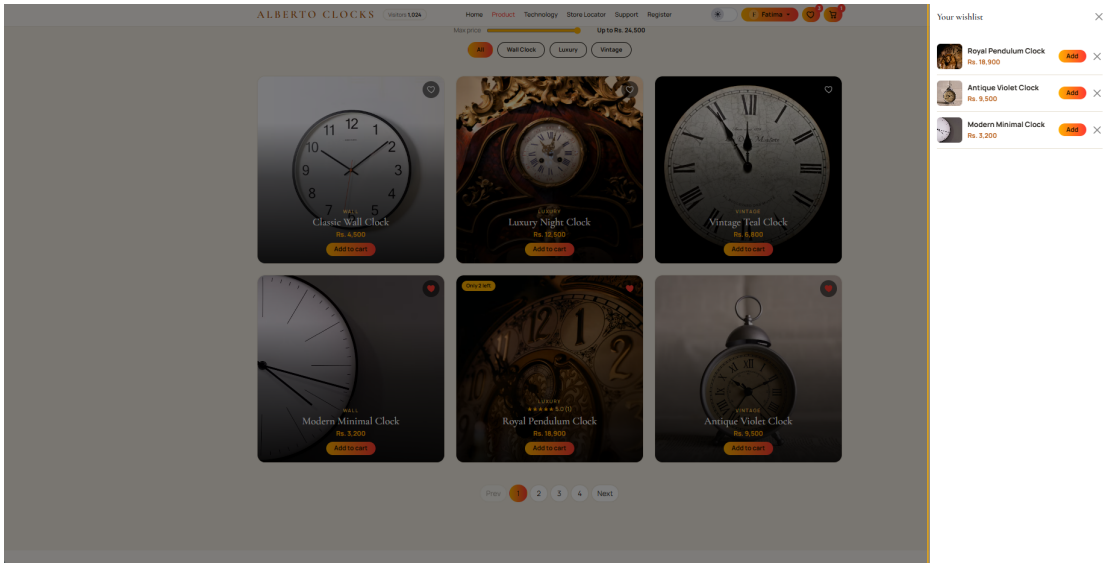


Figure 6: *Wishlist panel*

10.2 Reviews Module

The file `js/reviews.js` saves review information in browser storage. It calculates the average rating and the number of reviews for each clock.

A logged-in visitor can write or update a review. Review text is limited to 400 characters. The product page displays the available reviews and rating.

Reviews are saved in the current browser. The project does not check whether a reviewer purchased the clock, and saved review data can be changed by someone using that browser.

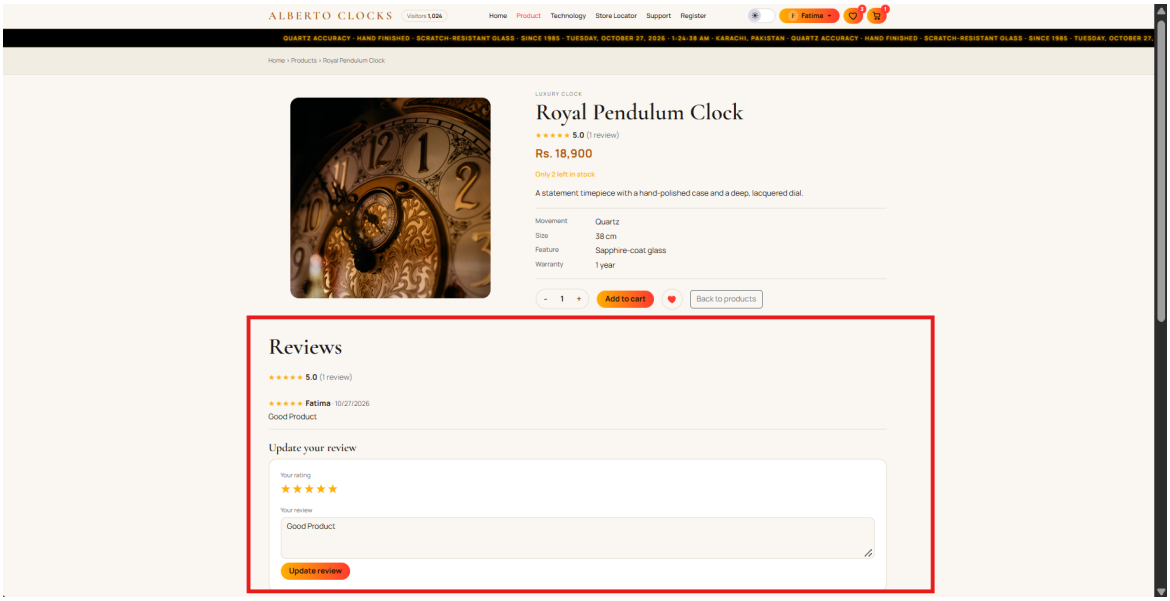


Figure 7: *Review list and rating form*

11. Login and Sign-Up

The Login window contains two tabs: Login and Sign Up. The sign-up form asks for full name, email, phone, gender, shipping address, password and password confirmation.

11.1 Registration Checks

Before saving an account, the code checks the password rules, compares the two password fields and checks whether the email is already registered in the current browser.

11.2 Password Rules

A valid password must:

- Contain 8 to 20 characters.
- Include an uppercase letter.
- Include a lowercase letter.
- Include a number.
- Include a special character.
- Contain no spaces.

The checklist updates as the visitor types. The eye button shows or hides the password.

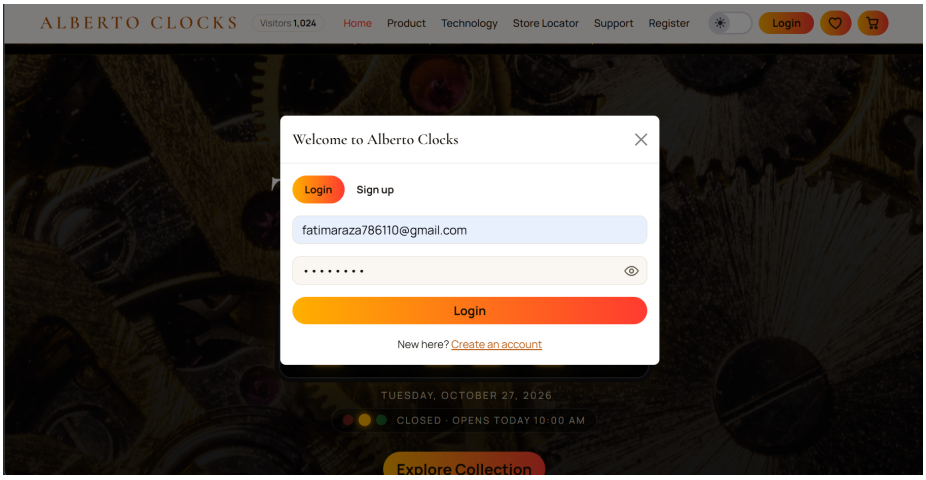


Figure 8: *Login window*

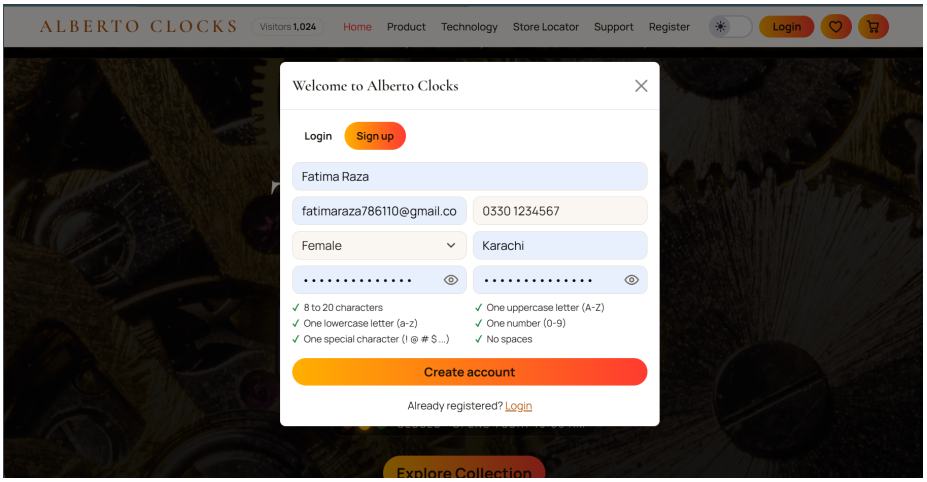


Figure 9: *Sign-up form*

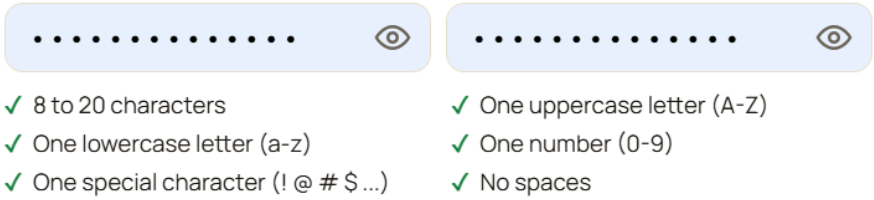


Figure 10: *Password checklist*

11.3 Successful Registration

The registration function checks the entered values, creates a salt and password hash for the demonstration account, saves the account in browser storage and signs the visitor in. The user's name then appears in the header.

11.4 Incorrect Login Details

The login form checks the entered email and password against the saved account. If the details do not match, an error message appears. The visitor can correct the details and try again.

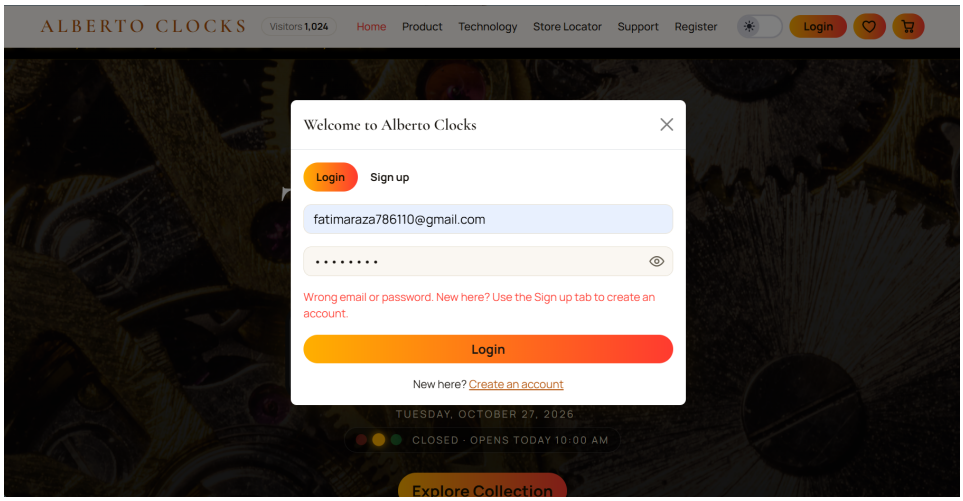


Figure 11: *Incorrect login message*

11.5 Password Error

If the password does not meet all the rules, the visitor must correct it before creating the account. The confirmation field must contain the same password.

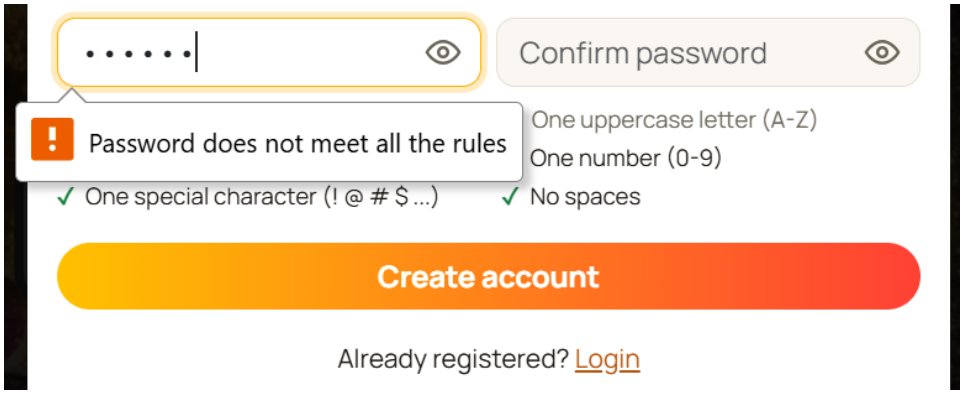


Figure 12: *Password validation message*

11.6 Logging Out

The Log Out action clears the active user in browser storage and returns the header to the Login button. It does not delete the saved account.

11.7 Account Limitation

The account feature is a browser-based demonstration. Its information is saved in the current browser and can be cleared or changed there. It is not shared with other devices. Do not use real passwords or sensitive personal information when testing a public copy of the project.

12. Profile and Order History

The profile displays the signed-in user's saved details and orders linked to that user's email. The profile and order history are shown separately.

12.1 My Profile

The profile displays the user's name, email, phone, gender, address and other saved account details. The Log Out button ends the active login session in the current browser.

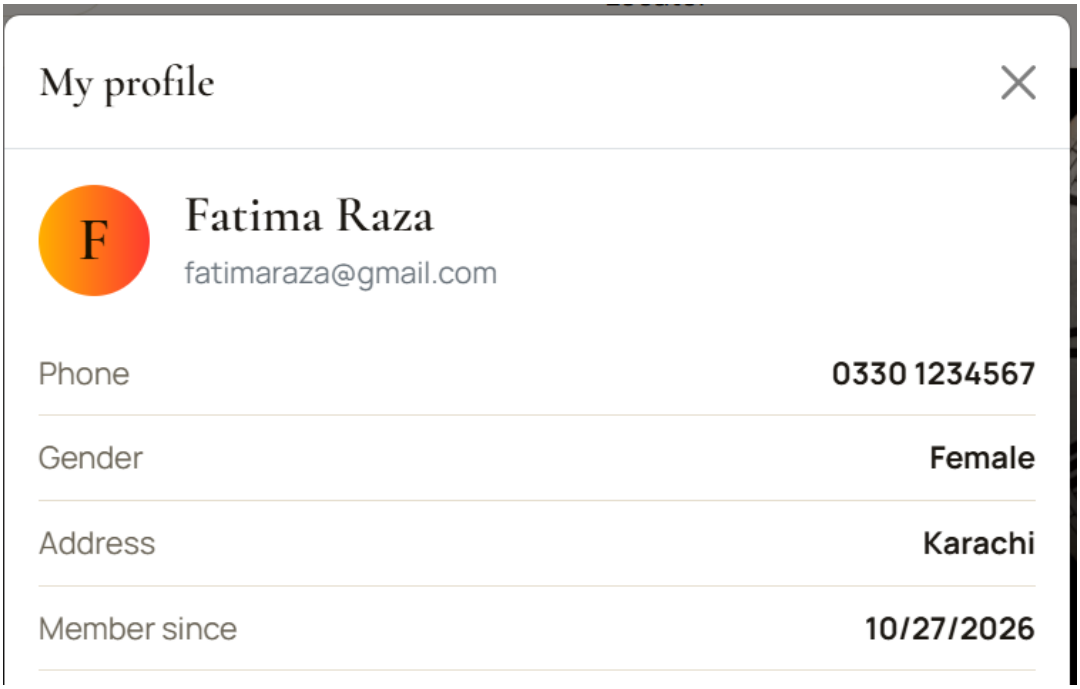


Figure 13: *User profile and account details*

12.2 Order History

Orders made through the demonstration checkout appear in the order history. Each order can show its order number, date, items, total and progress status.

Delivery orders move through Placed, Packed, Shipped and Delivered. Pickup orders move through Placed, Packed and Ready for pickup. These status changes are for demonstration only and are not linked to a courier.

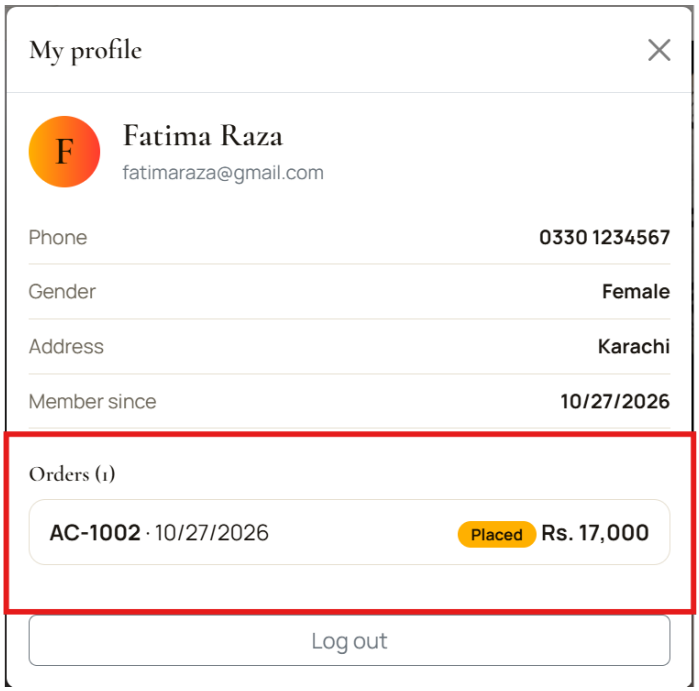


Figure 14: *Order history and order status timeline*

13. Shopping Cart and Coupon

The file `js/cart.js` manages the cart items and quantities. Each cart line contains a product ID and a quantity.

13.1 Adding a Product

When a visitor adds a clock, the code checks that the product exists and has stock remaining. The cart count and total are then updated.

13.2 Changing Quantity

The plus and minus controls change the quantity. An increase above the remaining stock is rejected. Removing a line updates the cart count and total. The Clear button empties the cart.

13.3 Coupon

The demonstration coupon is `TIME10`. The entered code is changed to uppercase before checking. A correct code applies the configured 10 percent discount. An incorrect code displays an error message.

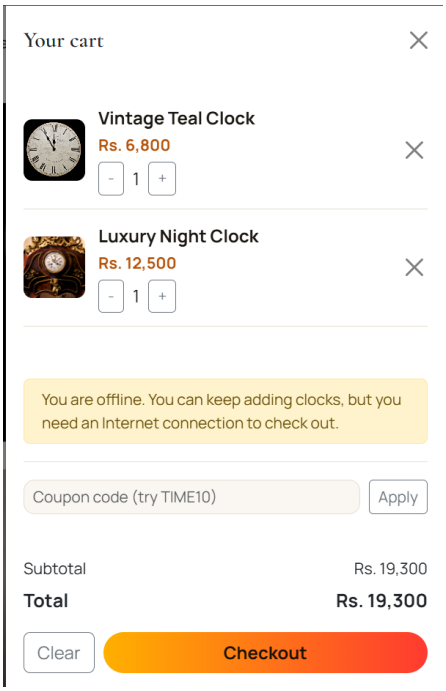


Figure 15: *Shopping cart with items and quantity controls*

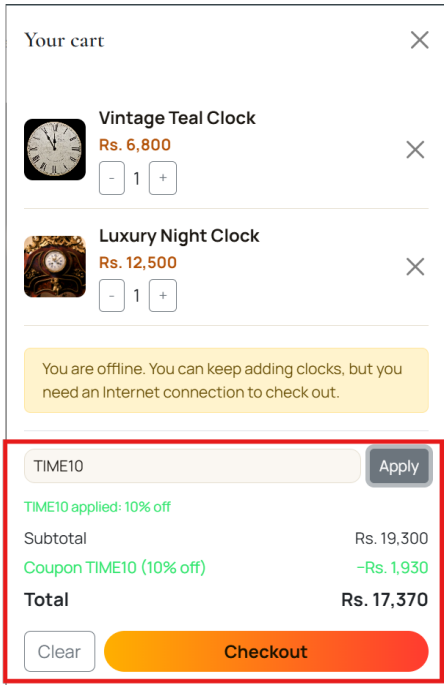


Figure 16: *Coupon field and updated total*

14. Checkout Implementation

The checkout handler checks whether the cart is empty, checks the Internet connection and asks the visitor to log in if needed. When the checks pass, the checkout form opens.

14.1 Checkout Fields

The form collects:

- Customer name.
- Phone number.
- Delivery address.
- City.
- Optional order note.
- Delivery or pickup preference.

It offers cash on delivery or showroom pickup. Address and city are required for delivery but not for pickup.

14.2 Phone Validation

The phone field accepts an optional plus sign and 10 to 15 characters made from digits, spaces and hyphens. If the value fails the check, an error appears and the order is not placed.

14.3 Saving an Order

When the form is valid, the code saves a local order number, customer details, item information, totals, coupon, payment preference and date. It updates the saved stock values and empties the cart.

The order number is based on the order list saved in this browser. It is not a shared store order number.

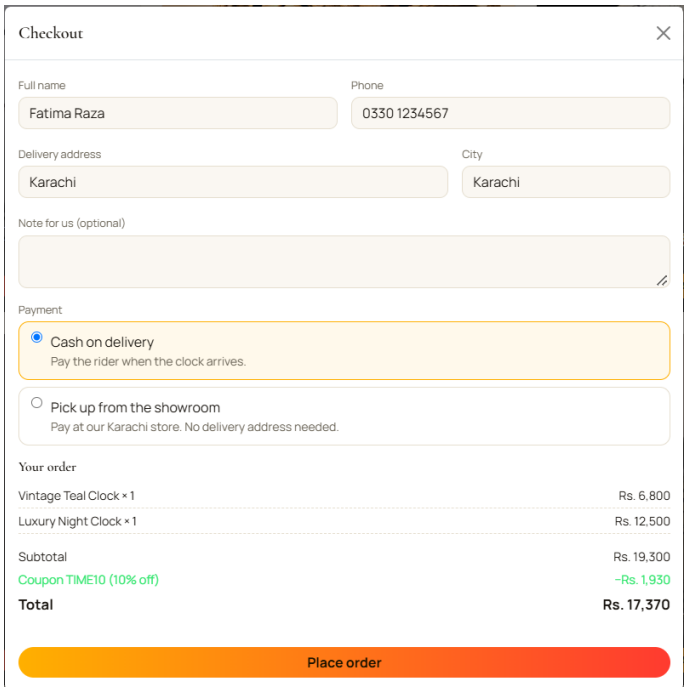


Figure 17: *Checkout form and order summary*

14.4 Checkout Flow

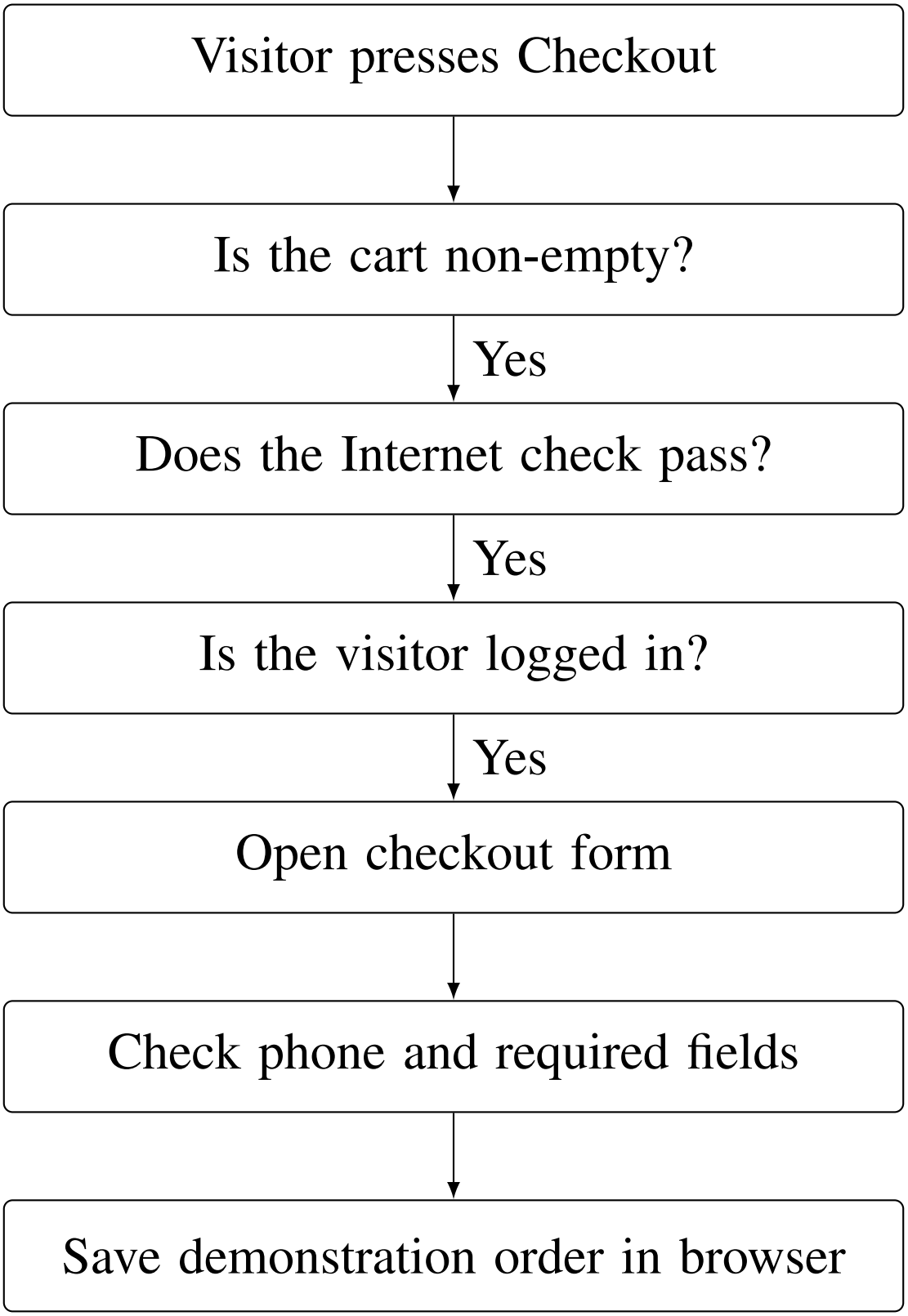


Figure 18: *Checkout steps*

14.5 Checkout Limitation

The checkout does not take payment and does not send an order to the store. The Internet check only checks connectivity. The order is saved in the current browser.

15. Clock, Ticker and Themes

The file `js/ui.js` controls the digital clock, date, ticker, world-time cards, themes, LED buttons, navigation, visitor counter and store status.

15.1 World Time

The world-time cards show Karachi, Dubai, London, New York and Tokyo. The times are formatted with JavaScript time-zone settings and updated in the browser.

15.2 Store Status

The displayed opening hours are Monday to Saturday, 10 am to 9 pm, with Sunday closed. The status is updated regularly, but it is not connected to a live store schedule.

15.3 Location

The browser may ask for location permission. If permission is granted, the ticker can show rounded coordinates. If permission is denied, the website continues using the default location label.

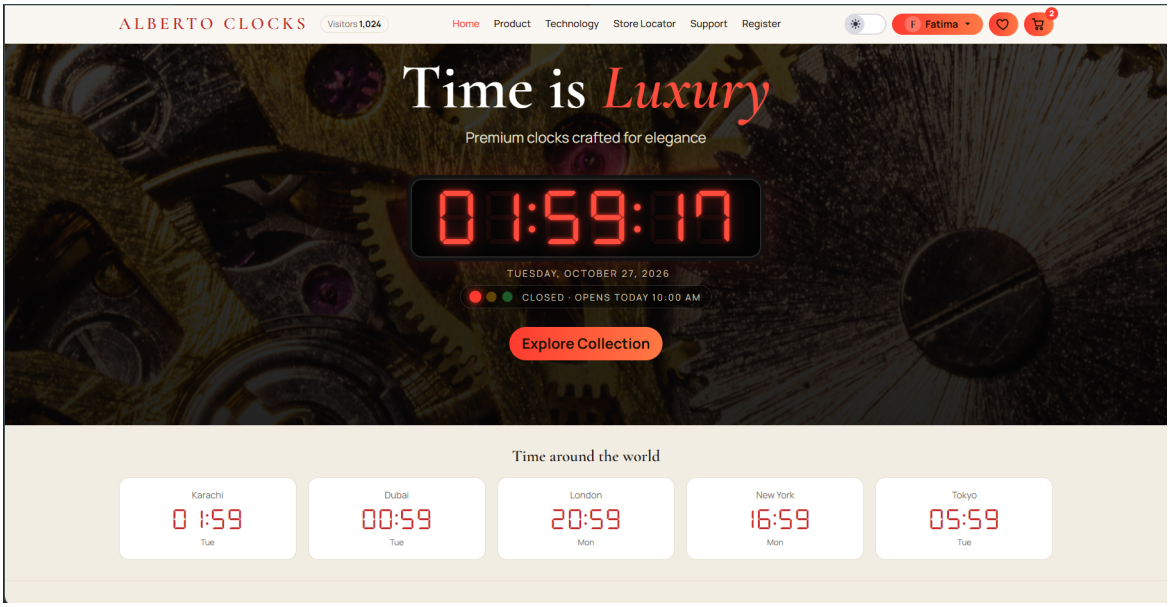


Figure 19: *Clock, ticker, themes and store status*

16. Support and Newsletter Forms

The file `js/forms.js` checks the required contact fields. When the visitor submits the form, the message is saved in browser storage and a confirmation is displayed.

The newsletter form checks the email format, clears the field and displays a confirmation. It does not create a real subscription or send an email.

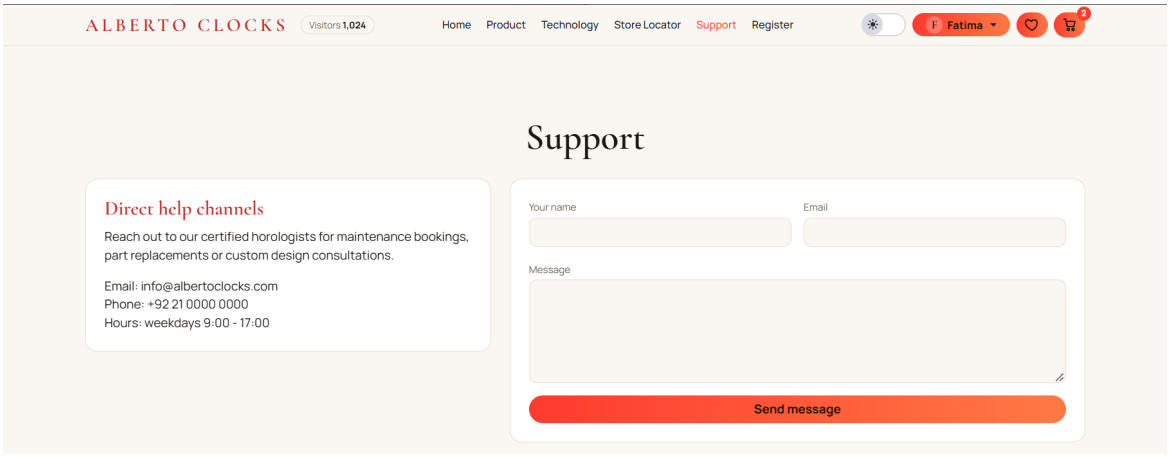


Figure 20: *Support section and contact form*

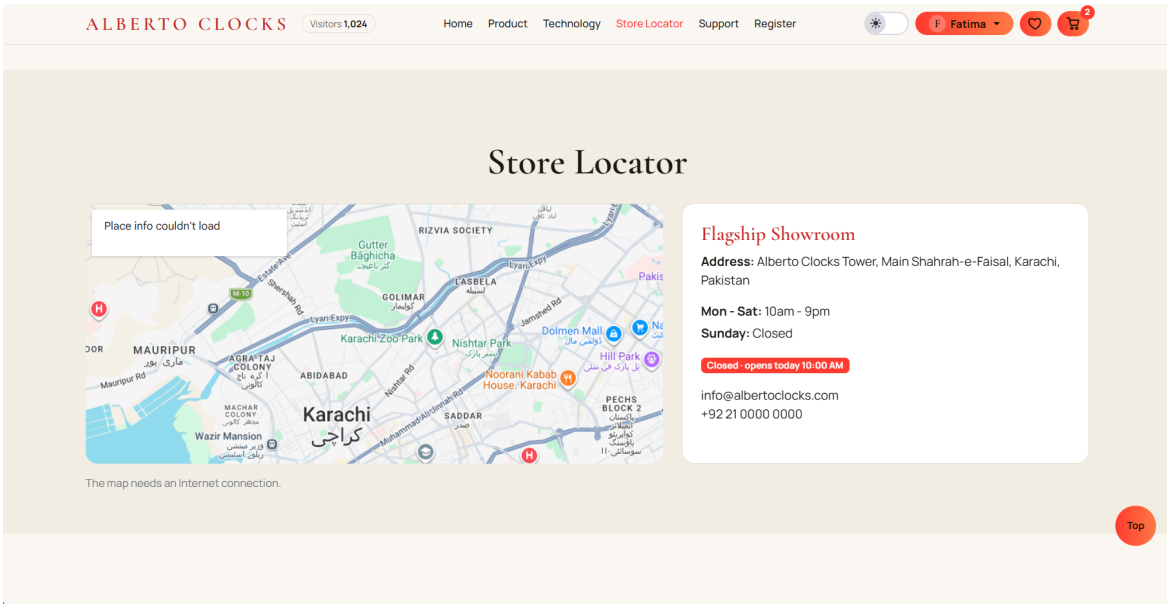


Figure 21: *Store Locator and map*

17. Browser Storage

The website uses browser storage to remember demonstration information between page reloads. The information belongs to the current website address in the current browser.

Key	Contents	Use
`ac_users`	Demo account records	Registration and login.
`ac_user`	Active user	Header and profile.
`ac_cart`	Product IDs and quantities	Restores the cart.
`ac_wishlist`	Saved product IDs	Restores wishlist items.
`ac_reviews`	Ratings and review text	Shows product reviews.
`ac_orders`	Demo orders	Profile order history.
`ac_sold`	Demo sold quantities	Stock changes in this browser.
`ac_coupon`	Current coupon	Cart discount.
`ac_messages`	Contact form entries	Keeps demonstration messages.
`ac_dark`	Theme preference	Remembers the theme.
`ac_led`	LED preference	Remembers the chosen LED colour.
`ac_visits`	Browser visit count	Stores the local visit count.
`ac_counted`	Session marker	Helps prevent repeat counting in the same session.

17.1 Resetting Demo Data

To clear test data, open the browser's developer tools and remove the website's local storage. This clears test accounts, orders, reviews, wishlist and cart. Only do this when the saved demo information is no longer needed.

18. Risks and Limitations

Area	**Risk**	**What to Remember**
Product data	JSON may fail to load.	Use Live Server and check the file path.
Images and map	Remote resources may not be available.	Keep fallback images and check Internet access.
Browser storage	Saved data can be changed or cleared.	Treat it as demonstration data only.
Account forms	Login is browser-based.	Do not use real passwords or sensitive information.
Stock	Stock changes only in this browser.	Visitors do not share the same stock count.
Checkout	No payment is taken and the order is not sent.	Treat orders as demonstrations.
Order status	Status changes after a short timer.	It is not a live delivery update.
Contact form	No actual email is sent.	Confirmation appears only on the website.
Location	The visitor may deny permission.	The rest of the site should still work.

19. Troubleshooting

Problem	**Possible Cause**	**Solution**
Product list is empty	JSON request failed.	Use Live Server and check the file.
Detail page says not found	Product ID is invalid.	Check the ID in the address and JSON.
Pictures are missing	Image address is unavailable.	Check the image address and fallback path.
Cart cannot add more items	Remaining stock is too low.	Check stock and saved sold quantity.
Coupon is rejected	Code was typed incorrectly.	Enter TIME10.
Login fails	Email or password is wrong.	Use the same browser and correct details.
Checkout reports offline	Connection check failed.	Connect to the Internet and try again.
Contact email does not arrive	The form is local.	No actual email is sent.
Saved data disappears	Browser storage was cleared or another browser is being used.	Use the original browser and avoid clearing site data.

20. Testing Checklist

Complete these checks before presenting the project.

ID	Test	Expected Result
T-01	Open the website through Live Server.	Home page loads.
T-02	Load the product catalogue.	Product cards appear.
T-03	Search for a known clock.	Matching clocks appear.
T-04	Search for a name with no match.	An empty-result message appears.
T-05	Change category and sort order.	Product list changes.
T-06	Move the maximum-price slider.	Price limit and product list update.
T-07	Open a product detail page.	Details for the selected ID appear.
T-08	Add a clock to the cart.	Cart count and total update.
T-09	Try to add a sold-out clock.	The item is not added.
T-10	Apply the TIME10 coupon.	Discount appears.
T-11	Start checkout while signed out.	Login is requested.
T-12	Register with an invalid password.	Password rules block submission.
T-13	Enter different password confirmation.	A mismatch message appears.
T-14	Log in with correct details.	Header displays the user's name.
T-15	Add and remove a wishlist item.	Wishlist updates.
T-16	Submit a product review.	Review appears in the browser.
T-17	Place a demonstration order.	Confirmation appears and cart empties.
T-18	Deny location permission.	Website remains usable.
T-19	Open the site on a narrow screen.	Navigation and cards fit the screen.
T-20	Submit the contact form.	Confirmation appears; no email is sent.

21. Maintenance and Future Improvements

21.1 Changing Product Information

Edit `data/products.json`. Use valid JSON, unique IDs, numeric prices, supported categories and valid image paths. Test the product card, details page, cart and wishlist after making changes.

21.2 Changing the Design

Place styles in the correct file under `css/`. Check both themes and test the layout on desktop and mobile screens.

The main CSS file is `css/style.css`. Keep its imports in the correct order.

21.3 Changing a Feature

Find the JavaScript file responsible for the feature. Change one feature at a time, test the normal steps and error cases, and check the browser console for errors.

21.4 Possible Future Improvements

- Connect contact messages to an email service.
- Connect checkout to a real payment service.
- Share stock and order information between visitors.
- Add a store-management screen.
- Add account recovery.
- Add more automated browser tests.

These are possible future improvements and are not part of the current project.

22. Conclusion

Alberto Clocks shows how HTML, CSS, JavaScript and JSON can be used to build a clock shopping website. The project keeps product information, styles, account forms, cart, wishlist, reviews and interface actions in separate files so that they are easier to understand and update.

The current website is a learning demonstration. Accounts, cart items, reviews, stock changes and orders remain in the current browser. Checkout, order status and support forms are not connected to live store services.

End of Developer's Guide